实验课 B Tensor and Tiny Vision Transformer

一、实验背景

张量是现代数值计算与深度学习系统中最基础的数据抽象. 无论是图像,序列还是中间特征，最终都可以表示为具有形状信息的多维数组. 掌握张量的存储方式,索引规则,形状变换与基础运算，对理解高性能数值程序设计,神经网络实现以及工程化推理系统都具有重要意义.

Vision Transformer 是近年来常见的视觉模型结构之一. 它将图像划分为若干 patch，并将 patch 序列送入 Transformer 编码器，从而建模图像不同区域之间的全局关系. 和卷积网络相比，Vision Transformer 在结构上更强调统一的张量表示,线性变换,注意力机制与模块组合.

本实验以 MNIST 手写数字识别 为任务背景，要求同学们先实现一个通用多维张量类，再基于该张量类构建一个 Tiny Vision Transformer 推理系统.

二、实验目标

阶段一：张量基础模块

要求实现一个可复用的通用多维张量类，作为后续网络层实现的底座.

1. 实现 模板类 `Tensor<T>`，支持张量数据与形状信息的统一封装.
2. 实现多维张量的基础操作，包括 shape 管理,索引访问,reshape,permute,transpose,unsqueeze,squeeze,slice,concat,softmax,matmul 等.
3. 张量的底层连续数据随机访问使用 `operator[]` 实现.
4. 保证张量对象具有良好的内存安全性，正确处理构造,析构,拷贝,移动与异常情况.

阶段二：tiny ViT 推理系统

要求基于第一阶段的张量类，实现一个用于 MNIST 手写数字识别的微型 Vision Transformer 推理框架.

1. 实现图像 patch 划分与 patch embedding.
2. 实现 Transformer 编码器中的核心模块，包括多头自注意力,前馈网络,LayerNorm,残差连接等.
3. 实现 `VisionTransformer` 整体结构，完成从输入图片到分类结果的前向推理.
4. 支持从文件中读取提供的模型配置与权重参数.

5. 输出最终分类结果，并支持导出某一层某一头的 attention 矩阵.

三、Tensor 与 Tiny ViT 工作原理

1. 多维张量基本工作原理

基本特性

- 张量可以看作带有 shape 信息的多维数组.
- 张量中的元素按照连续内存方式存储.
- shape 描述各维长度，决定索引展开与运算合法性.
- 多数运算都可归结为“读取张量 + 按规则变换 + 生成新张量”.
- 张量既是数据容器，也是后续神经网络各层交互的统一接口.

存储结构

- 张量内部通常包含三部分信息:
  - 数据区 `std::vector<T> _data`
  - 形状信息 `std::vector<size_t> _shape`
  - 步长信息 `std::vector<size_t> _strides`
- 若采用连续存储，则多维索引最终都会映射为一维下标.
- 张量应支持任意 rank，不应只局限于二维矩阵.

索引原理

- 对一个多维索引 `(i, j, k, ...)`，需根据各维 stride 映射到 `_data` 的一维位置.
- 若索引越界，应抛出异常.
- 本实验要求:
  - `operator[]` 用于访问底层连续存储数据中的第 `idx` 个元素.
  - `operator()(...)` 或 `at(...)` 用于多维坐标访问.

例如:

```
Tensor<float> x({2, 3, 4});
float a = x[5];            // 按底层一维存储访问
float b = x(1, 0, 2);      // 按多维坐标访问
```

基础运算

- **reshape**：仅改变 shape，不改变数据排列.
- **permute / transpose**：重排维度顺序.
- **unsqueeze / squeeze**：插入或删除长度为 1 的维度.
- **slice**：对某一维进行区间切片.
- **concat**：将多个张量按指定维拼接.
- **softmax**：沿指定维将一组实数映射为概率分布.
- **matmul**：对最后两维做矩阵乘法，前面的维度作为 batch 维处理.

2. Tiny ViT 基本工作原理

本实验中采用一个教学版的微型 Vision Transformer，用于 MNIST 手写数字识别.

输入表示

- MNIST 图片为灰度图，大小为 `1 × 28 × 28`.
- 将图片切分为若干 patch.
- 例如采用 `7 × 7` patch，则每张图会被切成 `16` 个 patch.

Patch Embedding

- 每个 patch 先被展平为一维向量.
- 然后通过一个线性层映射到固定维度 `hidden_dim`.
- 这样，每个 patch 就被转换为一个 token 向量.

加入 CLS Token 与位置编码

- 在 patch token 序列前增加一个可学习的 `CLS token`.
- 给每个 token 加上位置编码，使模型能够区分不同 patch 的位置.

若 patch 数为 `16`，则总 token 数为:

```
16 个 patch token + 1 个 CLS token = 17 个 token
```

attention 机制

在每个 Transformer 编码器块中，输入 token 序列会先经过线性映射生成:

- Query(Q)
- Key(K)

- Value(V)

随后计算注意力权重:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其含义是:

- 每个 token 会对所有 token 建立相关性评分.
- softmax 后得到权重分布.
- 再对 value 做加权求和，得到新的上下文表示.

多头注意力

- 多头注意力会将 hidden dim 划分到多个 head 上并行计算.
- 最后将多个 head 的结果拼接并线性映射回原维度.

推荐使用:

- reshape
- permute
- transpose
- matmul
- softmax

典型张量形状如下:

```
input x           : [B, T, H]
Q, K, V          : [B, T, H]
after reshape        : [B, T, num_heads, head_dim]
after permute        : [B, num_heads, T, head_dim]
scores           : [B, num_heads, T, T]
context          : [B, num_heads, T, head_dim]
```

前馈网络与残差结构

一个标准编码器块通常包含两部分:

1. 多头自注意力
2. 前馈网络

同时使用:

- 残差连接
- LayerNorm

其基本流程可写作:

```
x → MHA → Add → Norm → FFN → Add → Norm
```

分类输出

- 编码结束后，取 `CLS token` 的输出向量.
- 通过最后一个全连接层映射到 `10` 维.
- 输出 `0~9` 十个数字类别的 logits.
- 取 `argmax` 得到预测结果.

3. 实验配置

```
image size      = 28 x 28
patch size      = 7 x 7
num patches     = 16
num tokens      = 17
hidden dim      = 32
num heads       = 2
num blocks      = 2
mlp hidden dim  = 64
num classes     = 10
batch size      = 1
```

四、主要代码结构参考

文件名	作用
`Tensor.h`	负责声明并实现模板类 `Tensor<T>`，涵盖张量数据存储,形状管理,索引访问,reshape,permute,transpose,unsqueeze,squeeze,slice,concat,softmax,matmul等核心功能，为整个实验提供统一的底层数据结构与数值计算支撑.

文件名	作用
`Model.h`	负责声明 tiny ViT 推理系统相关的主要类，包括抽象层基类 `Layer`,线性层 `Linear`,归一化层 `LayerNorm`,图像切块嵌入 `PatchEmbedding`,多头注意力 `MultiHeadAttention`,前馈网络 `MLP`,编码器块 `TransformerBlock`,完整模型 `VisionTransformer` 以及辅助的权重加载类 `WeightLoader`，通过不同类完成模块划分与功能组织.
`main.cpp`	推理程序入口，负责读取输入图片,加载配置与权重,执行模型前向推理,输出分类结果

五、主要类设计

1. 张量类(Tensor)

- 模板参数:
  - `T`：张量元素类型，如 `float`,`double`
- 成员变量:
  - `_data`：底层连续存储
  - `_shape`：张量形状
  - `_strides`：各维步长
- 核心接口:
  - `size()`
  - `rank()`
  - `shape()`
  - `reshape(...)`
  - `reshaped(...)`
  - `permute(...)`
  - `transpose()`
  - `unsqueeze(...)`
  - `squeeze(...)`
  - `slice(...)`
  - `concat(...)`
  - `matmul(...)`

  - `softmax(...)`
  - `argmax()`
  - `operator[](size_t idx)`
  - `operator()(...)`

- 参考代码:

```
template <class T>
class Tensor {
private:
    std::vector<T> _data;
    std::vector<size_t> _shape;
    std::vector<size_t> _strides;

public:
    Tensor() = default;
    Tensor(const std::vector<size_t>& shape);
    Tensor(const std::vector<size_t>& shape, const T& init_val);

    size_t size() const;
    size_t rank() const;
    const std::vector<size_t>& shape() const;

    T& operator[](size_t idx);
    const T& operator[](size_t idx) const;

    template <class... Index>
    T& operator()(Index... idx);
    template <class... Index>
    const T& operator()(Index... idx) const;

    void reshape(const std::vector<size_t>& new_shape);
    Tensor<T> reshaped(const std::vector<size_t>& new_shape) const;
    Tensor<T> permute(const std::vector<size_t>& dims) const;
    Tensor<T> transpose() const;
    Tensor<T> unsqueeze(size_t dim) const;
    Tensor<T> squeeze(size_t dim) const;
    Tensor<T> slice(size_t dim, size_t begin, size_t end) const;
    static Tensor<T> concat(const std::vector<Tensor<T>>& tensors, size_t dim)
```

```
    Tensor<T> matmul(const Tensor<T>& other) const;
    Tensor<T> softmax(size_t dim) const;
    size_t argmax() const;
};
```

2. 抽象层基类(Layer)

- 功能:
  - 提供统一前向接口
- 核心接口:
  - `forward(const Tensor<float>& x)`
- 参考代码:

```
class Layer {
public:
    virtual Tensor<float> forward(const Tensor<float>& x) = 0;
    virtual ~Layer() = default;
};
```

3. 线性层(Linear)

- 功能:
  - 完成仿射变换
- 成员变量:
  - `_weight`
  - `_bias`
- 核心接口:
  - `forward(...)`

4. PatchEmbedding 类

- 功能:

  - 将 `1 × 28 × 28` 图片切分为多个 patch
  - 将 patch 展平后映射到 hidden dim

- 成员变量:
  - `_patch_h`
  - `_patch_w`
  - `_in_channels`
  - `_hidden_dim`
  - `_proj`

- 核心接口:
  - `forward(...)`

- 输出形状:

```
输入: [1, 28, 28]
输出: [1, num_patches, hidden_dim]
```

5. LayerNorm 类

- 功能:
  - 对 token 特征维做归一化
  - 提高数值稳定性

- 成员变量:
  - `_gamma`
  - `_beta`
  - `_eps`

- 核心接口:
  - `forward(...)`

- 说明:
  - 建议对输入张量最后一维进行归一化.

6. MultiHeadAttention 类

- 功能:
  - 实现多头自注意力
  - 完成 Q/K/V 投影,权重计算,head 拼接与输出投影
- 成员变量:
  - `_hidden_dim`
  - `_num_heads`
  - `_head_dim`
  - `_q_proj`
  - `_k_proj`
  - `_v_proj`
  - `_out_proj`
  - `_last_attention`
- 核心接口:
  - `forward(...)`
  - `get_attention_map()`
- 推荐实现流程:

```
x → q_proj/k_proj/v_proj
  → reshape to [B, T, H, D]
  → permute to [B, H, T, D]
  → scores = Q.matmul(K.transpose())
  → probs = softmax(scores)
  → context = probs.matmul(V)
  → permute  to [B, T, H, D]
  → reshape  to [B, T, hidden_dim]
  → out_proj
```

7. 前馈网络类(MLP)

- 功能:
  - 对每个 token 独立进行两层线性变换
  - 提升特征表达能力

- 成员变量:
  - `_fc1`
  - `_fc2`
- 核心接口:
  - `forward(...)`

8. Transformer 编码器块(TransformerBlock)

- 功能:
  - 组合多头注意力,LayerNorm,MLP 与残差连接
  - 作为 Vision Transformer 的基础重复模块
- 成员变量:
  - `_attn`
  - `_norm1`
  - `_mlp`
  - `_norm2`
- 核心接口:
  - `forward(...)`
- 推荐流程:

```
x → attn → residual add → norm1 → mlp → residual add → norm2
```

9. VisionTransformer 类

- 功能:
  - 组织完整推理流程
  - 负责 CLS token,位置编码,block 堆叠与分类输出
- 成员变量:
  - `_patch_embed`
  - `_cls_token`
  - `_pos_embed`

  - `_blocks`
  - `_head`

- 核心接口:
  - `forward(...)`
  - `predict(...)`
  - `export_attention(...)`

- 输入输出:

```
输入: [1, 28, 28]
输出: [1, 10]
```

10. WeightLoader

- 功能:
  - 从文件中读取提供的权重
  - 将参数加载到指定模块中
  - 对 shape 不匹配,文件缺失,格式错误等情况做异常处理

- 核心接口:
  - `load_tensor(...)`
  - `load_model(...)`